PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA

FACULTAD DE CIENCIAS E INGENIERÍA

ESCUELA DE INGENIERÍA EN COMPUTACIÓN Y TELECOMUNICACIONES

Práctica Teórico–Práctica de Recursividad

Análisis, transformación y rediseño de algoritmos

Asignatura: Algoritmos Clásicos y Estructuras de Datos

Frase guía de la práctica

No evalúa si puedes repetir una definición de recursividad; evalúa si puedes leer, desarmar, rediseñar y defender una solución cuando el problema cambia frente a ti.

Modalidad	Individual
Lenguaje	Java (solo librerías estándar)
Énfasis	Razonamiento, trazado mental, rediseño y análisis de complejidad

Propósito de la práctica

- Obligar al estudiante a pensar la recursividad como una estrategia de descomposición, no como una plantilla memorizable.
- Leer código sin ejecutarlo, reconstruir su árbol de llamadas y justificar cómo cambia su costo cuando la idea algorítmica cambia.
- Transformar soluciones recursivas en alternativas más eficientes mediante iteración, pila manual o memorización, según corresponda.
- Distinguir entre problemas con subproblemas repetidos y problemas donde cada llamada es estructuralmente necesaria.

Reglas de trabajo

- No se permite responder con definiciones genéricas. Toda afirmación debe conectarse con el código dado.
- Si propones una mejora, debes explicar exactamente qué se elimina: profundidad, llamadas repetidas, concatenaciones costosas o trabajo redundante.
- Puedes codificar las versiones pedidas, pero la nota principal está en el razonamiento, el trazado mental y la defensa de tus decisiones.

- No se permite usar colecciones de Java (ArrayList, Stack, HashMap, etc.) salvo que el enunciado lo autorice de forma expresa.
- No se permite copiar una solución de internet o generada por IA sin poder defender cada línea, cada caso base y cada cambio de complejidad.

Conexión con la teoría vista en clase

La práctica se apoya en las ideas trabajadas sobre caso base, progresión hacia el caso base, relación entre recursividad e iteración, costo temporal y espacial, y alternativas como pila manual y memorización. También retoma el contraste entre factorial, Fibonacci y Torres de Hanoi para obligarte a distinguir cuándo la recursividad solo es elegante y cuándo además es estructuralmente correcta.

Formato de entrega

- Un repositorio en GitHub con el código Java, organizado por ejercicio.
- Un README.md con instrucciones de compilación y ejecución, y pruebas propias.
- Un archivo PDF o Markdown con las respuestas razonadas a cada inciso.
- Esquemas del árbol de llamadas donde el enunciado los solicite.

Ejercicio 1 — Factorial: de recursión lineal a diseño consciente (20 puntos)

Código base:

```
public static long factorial(int n) {
if (n <= 1) {         return 1;
    }
    return n * factorial(n - 1);
}
```

Tareas:

1. Sin ejecutar el código, dibuje el árbol o la cadena de llamadas para factorial (5) e indique qué información queda pendiente en cada nivel de la pila.
   factorial(5)
   |___n*factorial(4)
   |___n*factorial(3)
   |___ n *factorial(2)
   |___ n * factorial (1) → 1
   Resultado: 120

2. Explique por qué esta función no genera un árbol ramificado como Fibonacci, sino una sola rama de profundidad n.
   Esto se debe ya que en el caso del factorial hace una sola llamada recursiva, sin embargo, en el caso del Fibonacci hace dos llamadas recursivas, una para el numero anterior a n y otra para el numero que esta en dos posiciones anteriores a nen la posición anterior a ese, por eso el factorial forma una rama única lineal mientras que el Fibonacci se ramifica

3. Escriba la recurrencia temporal T(n) y justifique, sin usar frases vacías, por qué su complejidad temporal es lineal y su complejidad espacial también es lineal.
   Su complejidad temporal es lineal ya que trabaja por llamada constante, la comparación del caso base de n <= 1, y si no es el caso base multiplica y hace una llamda, en notación Big O esto es constante. Su complejidad espacial también es constante ya que cada llamada de la función crea un frame que es de tamaño constante, entonces la profundidad de la recursión es n en el peor caso hasta llegar al caso base que es 1, entonces en resumen tanto la complejidad temporal como la espacial son lineales por crecimiento de la pila.

4. Reescriba el algoritmo en una versión iterativa y compare formalmente qué costo desaparece al eliminar la recursión.

```
public static long factorialIterativo (int n) {
   long resultado = 1L;
   for (int i = 2; i <= n; i++) {
      resultado *= i;
   }
   return resultado;
}
```

   Con la version iterativa con respecto al tiempo el bucle ejecuta una cantidad de multiplicaciones de n-1, por lo que su complejidad es de O(n), mientras que el espacio de las variables resultado para retornar el resultado y la variable definida en el bucle for i es constante de O(1) por lo que no crece la pila

   Por ello en esta nueva versión se elimina la creación de frames por cada llamada, que en el caos de la recursiva lo hacia para la direcciones de retorno, parámetros…, además desaparecen los saltos de retorno y la espera de la llamada del stack en cada paso, entonces en tomando en cuenta la complejidad espacial, la forma recursiva que es de O(n) se reduce a O(1) en la forma iterativa, sin embargo el tiempo exacto sigue siendo lineal O(n), pero con menor constante por evitar la sobrecarga de llamadas que puede generar la forma recursiva

5. Diseñe una versión de recursividad de cola con acumulador y argumente por qué en Java sigue sin garantizarse una optimización real del stack.

```
public static long factorialCola(int n) {
   return factorialColaAux(n, 1L);
}

private static long factorialColaAux(int n, long resultado) {
   if (n <= 1) return resultado;
   return factorialColaAux(n - 1, resultado * n);
}
```

   Para esta versión de recursividad de cola con acumuladora se hace que la última operación de la función sea la llamada recursiva, así se lleva el resultado que hay parcialmente a un acumulador. La ventaja de esta versión es que en algunos lenguajes el compilador puede reusar el mismo frame logrando una complejidad constante de O(1) como en la versión iterada pero esto es para los lenguajes con optimización de cola de llamada, lo cual en java no exige esto por lo que los jvm no hacen eliminación general de las llamadas en cola, entonces la pila

seguirá creciendo con la recursión y puede desencadenarse en un stackoverflow para n grandes, por lo tanto no garantiza en java la optimización del stack, el espacio sigue siendo lineal

6. Explique en qué contexto práctico seguiría prefiriendo la versión recursiva, aun sabiendo que la iterativa usa menos memoria.

   Aunque la iterativa usa menos memoria, hay contextos donde la recursión es **mejor,** para funciones que su naturaleza es recursiva como el factorial, la recursión facilita la verificación y el razonamiento, además en arboles la recursividad refleja la estructura de la búsqueda por profundidad mejora la legibilidad del código, también en algoritmos de tipo divide y vencerás como la búsqueda binaria, que divida un problema en subproblemas y lo resuelve por separado, la recursión ayuda al desdoblamiento del problema, además a entradas pequeñas la recursión no presenta riesgo de desborda la pila

Lo que se está evaluando aquí
No basta con decir "es O(n) porque se llama n veces". Debes mostrar de qué manera el problema se reduce, qué trabajo constante ocurre por llamada y por qué la pila crece exactamente con la profundidad de la recursión.

Ejercicio 2 — Fibonacci: detectar el verdadero origen del costo (25 puntos)

Código base:

```
public static int fib(int n) {
if (n <= 1) {         return
n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

Tareas:

7. Construya manualmente el árbol de llamadas de fib(6). No se acepta solo el resultado final; debe marcar cuáles subproblemas se repiten.

```
Fib(6)
|___fib(5)
|     |___fib(4)
|     |      |__fib(3)
 |    |      |     |___fib(2)
|      |     |    |___fib(1) -> 1
|     |       |__fib(2) --- Repetido
|     |           |___fib(1)->1  --- Repetido
|     |
|     |___fib(3)   ---Repetido
|           |___fib(2)   -- Repetido
|           |      |___fib(1) -> 1  ---Repetido
|           |___fib(1)->1     --- Repetido
|
```

```
|___fib(4) ---- Repetido
    |___fib(3)  -- Repetido
    |     |__fib(2)    ---Repetido
    |         |___fib(1) -> 1  --Repetido
    |___fib(2) --- Repetido
          |___fib(1) -> 1   ---Repetido
```

Resultado final: 720

8. Explique por qué el costo de este algoritmo no crece por tener recursión, sino por recalcular subproblemas ya resueltos.
   La recursión en si no causa que la complejidad sea alta por ejemplo, el factorial recursivo es O(n). El problema es que el fibonacci es ingenuo, el algoritmo vuelve a resolver el mismo subproblema muchísimas veces. Por ejemplo, para fib(6) se calcula fib(4) dos veces, para el fib(3) se calcula fib(2) dos veces y así sucesivamente, esto genera que se ramifique donde cada nivel duplica el trabajo anterior, el verdadero origen del costo esta en la cantidad de subproblemas repetidos, no en las llamadas recursivas en si

9. Señale al menos tres llamadas redundantes dentro de su árbol y explique cómo esas repeticiones empujan el tiempo hacia crecimiento exponencial.
   Las **l**lamadas redundantes dentro del árbol de fib(6) son por ejemplo la de fib(4) que se calcula dos veces, donde cada calculo genera su propio subárbol, la de fib(3) que se calcula 4 veces y la de fib(2) que se calcula 8 veces, entonces el costo es empujado a ser exponencial porque estas repeticiones se multiplican en cada nivel , en el primer nivel es una llamada, en el segundo nivel son 2, en el tercero 4 y en el ultimo nivel son 8 llamadas, cada nivel de abrol duplica el numero de nodos del anterior por lo tanto la complejidad es exponencial de O(2^n)

10. Analice cómo cambia la complejidad temporal y espacial cuando pasa de la versión recursiva ingenua a la memorizada (ver teoría).

La complejidad temporal para la versión recursiva ingenua sin memorización es exponencial, O(2^n), por el árbol completo y el espacio espacial es lineal , O(n), ya que es la profundidad máxima de la recursión, sin embargo, para la versión con memorización ya como aquí se guarda cada resultado del fibonacci después de calcularlo la primera vez la complejidad temporal mejora ya que solo se calcula una vez los Fibonacci de cierto numero y cada calculo hace solo un trabajo constante por lo que la complejidad temporal se reduce a ser lineal de O(n), entonces cambia de exponencial a lineal, mientras que la complejidad espacial se mantiene de forma lineal de O(n), ya que el espacio total será el del arreglo para la memorización más la pila recursiva que es lineal también. La memorización elimina la repetición de subárboles completos, lo que colapsa el árbol exponencial a una simple cadena lineal.

11. Explique por qué la tabulación iterativa en este caso no solo cambia la implementación, sino también la forma de pensar el problema: de arriba hacia abajo vs. de abajo hacia arriba.

    Esto se debe a que en la tabulación iterativa ya no se piensa en resolver el Fibonacci de un numero n, sino el construir el Fibonacci por ejemplo de 0 y 1 luego se usa eso para construir el de 2 luego el de 3 y así sucesivamente hasta llegar al Fibonacci de n. Esto invierte el pensamiento ya que la recursión de arriba abajo parte del gran problema y bajo dividiendo el problema, los subproblemas se repiten y es exponencial, mientras que en la tabulación iterativa de abajo a arriba se construye la solución, se parte de los casos bases y subo, cada subproblema se calcula una vez y la complejidad es lineal
    .

12. Defienda cuál de las tres versiones usaría si el programa debe responder 100,000 consultas de Fibonacci con distintos valores de n en una misma ejecución.

    Para 100,000 consultas, la mejor opción es la tabulación de abajo a arriba, esta construye todos los valores hasta el máximo n una sola vez y cada consulta es constante, además elimina la recursión por lo que no hay costos de stack, no hay peligro de stackoverflow y el acceso a los valores es directo, asimismo desaparecen todas las repeticiones del algoritmo ingenuo y las repeticiones entre consultas y las repetidas dentro de una misma consulta.

Restricción intencional anti-respuesta genérica
No se aceptará "la solución iterativa es mejor porque es más rápida". Debes identificar qué repeticiones desaparecen, qué estructura guarda resultados previos y cómo eso altera el crecimiento del algoritmo.

Ejercicio 3 — Torres de Hanoi: cuando la explosión no es un defecto (30 puntos)

Código base:

```
public static void hanoi(int n, char origen, char auxiliar, char destino) {
if (n == 1) {
        System.out.println(origen + " -> " + destino);
return;
    }
    hanoi(n - 1, origen, destino, auxiliar);
System.out.println(origen + " -> " + destino);     hanoi(n
- 1, auxiliar, origen, destino);
}
```

Tareas:

13. Para n = 3, trace a mano la secuencia completa de llamadas y movimientos. Luego explique por qué el movimiento central del disco más grande divide la solución en dos problemas simétricos.

    hanoi(3, A, B, C)

    |___ hanoi(2, A, C, B)

    |____hanoi(1, A, B, C) → A -> B

    └── A -> C

    |—— hanoi(1, B, A, C) → B -> C

    ├— A -> C

    |— hanoi(2, B, A, C)

├── hanoi(1, B, C, A) → B -> A

├── B -> C

└── hanoi(1, A, B, C) → A -> C

Secuencia del movimiento: A → B**,** A → C **,** B → C **,** A → C (movimiento del disco mayor), B → A**,** B → C**,** A → C

Para mover el disco más grande que es el 3, se debe de primero mover todos los discos pequeños 1 y 2 al poste auxiliar, eso es hanoi(2), hacer una sola jugada del disco más grande y luego mover esos mismos discos pequeños desde el auxiliar al destino, eso vuelve a se hanoi(2), dos subproblemas idénticos alrededor del movimiento del disco mayor

14. Escriba la recurrencia M(n) del número mínimo de movimientos y resuélvala hasta llegar a la forma cerrada.

De acuerdo al razonamiento anterior:

La recurrencia que describe el número mínimo de movimientos necesarios para resolver las Torres de Hanoi con n discos es M(n) = 2M(n-1)+1, porque para mover el disco más grande primero se deben mover los n-1 discos superiores, luego hacer un único movimiento central y finalmente volver a mover esos mismos discos. Al resolver esta recurrencia por expansión repetida, se observa que cada paso duplica el término anterior y suma 1, lo que genera una serie geométrica que finalmente conduce a la fórmula cerrada M(n) = 2^n -1, esta expresión indica que el crecimiento del número de movimientos no es arbitrario, sino exactamente exponencial y determinado por la estructura del propio problema.

15. Explique por qué Hanoi también tiene crecimiento exponencial, pero por una razón distinta a Fibonacci: aquí no hay recomputación inútil, sino que cada llamada representa trabajo estructuralmente necesario.
En Fibonacci el crecimiento exponencial ocurre por repetición de subproblemas, el árbol se expande porque se vuelven a calcular las mismas cosas innecesariamente. En hanoi el crecimiento exponencial ocurre porque cada movimiento es indispensable físicamente, no es que sea una mala implementación, no hay repeticiones y no hay trabajo desperdiciado, a diferencia del Fibonacci en el hanoi para mover el disco n, todos los discos menores deben estar fuera del camino, eso exige mover n-1 discos al poste auxiliar, luego se mueve el disco grande luego mover otra vez esos n-1 discos al poste destino, por lo que es imposible omitir algún movimiento, cada uno es obligatorio, por eso en Fibonacci el problema es recomponer y en el hanoi es inevitable

16. Argumente por qué no existe una versión iterativa trivial que mejore la complejidad del problema. Puede cambiar la forma de implementarlo, pero no el número mínimo de movimientos requerido.

Se puede cambiar cómo se genera la secuencia de movimientos de iteración, pila,…., pero no podemos reducir el número mínimo de movimientos, las reglas de este algoritmo son no poner un disco grande sobre uno pequeño y que solo un disco se mueve por vez, por ello cualquier algoritmo, recursivo o no, requiere al menos 2^n – 1 movimientos, por esto cambiar la implementación no reduce el tiempo, el costo esta en la propia tarea en la forma de como se programa

17. Diseñe una estrategia iterativa o con pila manual que simule la recursión. No se evalúa por ser más rápida, sino por demostrar que entendiste qué información guarda cada marco de llamada.
Para simular la recursión manualmente, basta almacenar en la pila la información del frame, cada frame necesita un n, un origen, un auxiliar y un destino, asimismo necesitara un indicador sobre que parte del procedimiento faltaría por ejecutar, el algoritmo seria muy similar al recursivo en términos de pasos y complejidad, mostrando que la recursividad solo es una forma de expresar el estado

18. Responda: ¿qué cambia si el objetivo del programa no es imprimir movimientos, sino únicamente contar cuántos se necesitan? Rediseñe la solución y compare sus costos.
    Si el objetivo del programa deja de ser imprimir cada movimiento y pasa a ser solo contar cuántos se necesitan**,** el problema se vuelve muchísimo más simple, entonces y a no es necesario ejecutar toda la secuencia ni simular el proceso de mover discos, basta con usar la fórmula matemática del número mínimo de movimientos de 2^n -1, esto transforma el costo de un algoritmo exponencial que imprimía cada uno de los movimientos en un cálculo instantáneo de tiempo constante y espacio constante. En otras palabras, imprimir obliga al programa a realizar físicamente cada paso, mientras que contar solo requiere evaluar la fórmula, lo que elimina por completo el crecimiento exponencial del tiempo y evita el uso de recursión profunda o estructuras adicionales.

19. Por qué Hanoi es un excelente contraejemplo para el estudiante que cree que si una recursión es O(2^n), entonces está mal diseñada. Porque Hanoi no es exponencial por mala programación sino porque la naturaleza del problema es exponencial, no hay una solución sub-exponencial, es el mínimo matemáticamente, el rendimiento del algoritmo no solo depende de la forma de programar sino también de como sea el problema, ya uqe puede ser recursivo y exponencial sin estar mal diseñado, la complejidad no proviene de repetir trabajo sino de la naturaleza inevitable del trabajo requerido

Pista conceptual
En Fibonacci la pregunta clave es “qué se repite”. En Hanoi la pregunta clave es “qué es inevitable”. Si confundes esas dos ideas, tu análisis quedará incompleto.

Ejercicio 4 — Mutaciones de código: cambiar la complejidad con cambios mínimos (25 puntos)

Analice cada fragmento. En todos los casos debe: identificar el costo dominante, proponer un cambio de diseño y justificar exactamente por qué cambia la complejidad.

Fragmento A:

```
public static int potencia(int x, int n) {
if (n == 0) return 1;     return x *
potencia(x, n - 1);
}
```

Fragmento B:

```
public static String invertir(String s) {
if (s.length() <= 1) return s;
    return invertir(s.substring(1)) + s.charAt(0);
}
```

Fragmento C:

```
public static int contarUnos(int[] a, int i) {
if (i == a.length) return 0;
    return (a[i] == 1 ? 1 : 0) + contarUnos(a, i + 1);
}
```

Tareas:

20. En el fragmento A, reemplace la idea recursiva lineal por exponenciación rápida. Debe justificar por qué el problema deja de reducirse de n en n y pasa a reducirse por mitades.

```
public static String invertirIterativo(String s){
StringBuilder sb = new StringBuilder(s.length());
   for (int i = s.length() - 1; i >= 0; i--) {
      sb.append(s.charAt(i));
   }
   return sb.toString();
}
```

La solución reduce de n a n-1 en cada llamada ya que hace n multiplicaicones y n frames en la pila, la complejidad es temporal lineal de O(n) y la espacial también, esta cambia ya que el problema deja de reducirse y pasa a partirse por la mitad en cada paso, entonces el numero de niveles de recursión es de logarítmica con un tiempo y espacio logaritmico

21. En el fragmento B, explique por qué la lógica parece lineal, pero puede degradarse por el costo acumulado de concatenaciones y subcadenas. Luego proponga una versión que reduzca ese costo.

```
public static int contarUnos(int[] a, int i) {
   if (i == a.length) return 0;
   return (a[i] == 1 ? 1 : 0) + contarUnos(a, i + 1);
}
```

Aquí aunque se hacen n niveles de recursión cada concatenación crea un nuevo string copiando caracteres por lo que cada llamada construye una subcadena nueva, el efecto será que la suma de copias de caracteres en concatenaciones y subcadenas conduce a un costo acumulado de un tiempo cuadratico y una pila lineal. La complejidad cambia porque desaparece el costo cuadratico de concatenación y de crear subcadenas en cada nivel

22. En el fragmento C, argumente por qué cambiarlo a iterativo mejora el espacio pero no el tiempo asintótico. Luego responda: vale la pena cambiarlo siempre.

```
public static int contarUnosIter(int[] a) {
   int count = 0;
   for (int i = 0; i < a.length; i++) {
      if (a[i] == 1) count++;
   }
   return count;
}
```

En este se recorre el arreglo una vez con un tiempo lineal y en cada llamada agrega un frame de espacio lineal , el tiempo será linela al igual que en la versión recursiva ya que no hay forma de contar cada elemento sin mirar y de un espacio constante ya que desaparece el uso de pila, si vale la pena cambiarlo cuando el tamaño de la entrada puede crecer y así evitar un overflow, además no es estrictamente necesario si el arreglo es muy pequeño o se busca claridad didáctica, aun así la versión iterativa es preferible ya que mantiele eel mismo tiempo lineal y reduce el espacio a constante

Criterios de evaluación

Criterio	Qué se espera	Pts.
Trazado y lectura del comportamiento	Reconstruye correctamente llamadas, estados pendientes y estructura del problema sin depender de ejecutar el programa.	20
Argumentación de complejidad	Relaciona código, recurrencia y crecimiento real; evita respuestas tipo porque tiene dos llamadas.	25
Rediseño algorítmico	Propone cambios que realmente modifican el origen del costo y los justifica con claridad.	25
Implementación en Java	Código correcto, consistente, sin atajos prohibidos y alineado con el análisis.	15
Claridad de la defensa escrita	Respuestas propias, conectadas con el problema y técnicamente defendibles.	15

Nota importante

Advertencia al estudiante

Una respuesta puede tener la notación correcta y aun así perder puntos si no demuestra por qué esa notación describe el comportamiento del algoritmo. En esta práctica el razonamiento vale más que la etiqueta.